PX

Practice papers

Three full papers written in this lecturer's style, using what was said in the exam briefing.

INTE 22313
4 × 25 MARKS

P1
P2
P3

What these papers are built on

- **All four questions are case-based**, and a single case asks about **two or more patterns**, spread across the whole question rather than saved for the last part (exam briefing).
- The shape of the past papers: identify the patterns with reasons → diagram → short Java → a follow-up part (challenges, HA/DR, SOLID, how two patterns connect).
- Question 2 is microservices in every paper, because 2023, 2024 and 2025 all had it.
- Scenarios the past papers haven't used yet: food delivery, smart home, online exams, ride-hailing, insurance, banking, EV charging, airline, hotel, returns.

How to practise

do it like the real thing

1. **Time it.** 2½ hours for 4 questions. Give each question about **35 minutes** and move on when the time is up, even mid-answer. The lecturer's biggest warning: students write long first answers and run out of time for the last ones.
2. **Use only what you'll take in:** the printed exam note and the past-papers booklet. The point is to check whether you can find things fast. Mark any page you had to hunt for.
3. **Start from the case, not the theory.** First sentence: what the case needs ("The kiosk behaves differently in each state, so..."). Then name the pattern and give reasons tied to the case. Don't write everything you know about the pattern.
4. **Write something for every part.** A name plus one reason earns more than a blank.
5. **Mark yourself** with the answers booklet. Note what you missed, not just whether the pattern name was right.

Clock for each paper

TIME	DO
0:00 – 0:05	Read all four cases. Underline the problem words. Write assumptions.
0:05 – 0:40	Question 1
0:40 – 1:15	Question 2 (microservices: use the kit in tab 11, it's fast)
1:15 – 1:50	Question 3
1:50 – 2:25	Question 4
2:25 – 2:30	Fill any blank part with at least a pattern name and one reason.

Short on time tonight? Do Paper 1 fully under time. For Papers 2 and 3, spend 5 minutes per question: write only the pattern names, one reason each, and which diagram and code you would draw. Then check against the answers.

1 Paper 1 · Question 1

Food delivery app

PRACTICE PAPER 1
25 MARKS

PRACTICE PAPER 1

INTE 22313 – Software Design Patterns and Frameworks · No. of questions: 4 · Time: 2½ hours
Answer ALL questions. Allocate time by the marks. State any assumptions you make. This is an OPEN BOOK exam.

QUESTION 1 · 25 MARKS

QuickBite is a food delivery app. A customer's order moves through these stages: **Placed** → **Confirmed** by the restaurant → **Preparing** → **Out for delivery** → **Delivered**. An order can be **Cancelled** only before it is out for delivery. What a customer can do (cancel, track, rate) depends on the current stage.

QuickBite works with many restaurants. Each restaurant already uses its own point-of-sale system from a different vendor: one accepts orders through `pushOrder(xml)`, another through `createTicket(json)`. QuickBite cannot change these systems.

When a customer taps "Place order", the app must check the restaurant, take the payment, assign a rider and send notifications. The mobile team wants one simple call for this instead of talking to four services.

1.1 Identify a suitable design pattern, with reasons, for each of: (a) the order stages, (b) sending orders to the restaurants' different systems, (c) the single "Place order" call. — 6 marks

1.2 Draw a state diagram for an order. — 5 marks

1.3 Write Java for the OrderState interface and the Order context class. Show one concrete state. — 6 marks

1.4 Draw a class diagram showing how QuickBite connects to two restaurants' point-of-sale systems. — 5 marks

1.5 Give one challenge of the "Place order" design in (c) and how you would overcome it. — 3 marks

2 Paper 1 · Question 2

Hospital management system

PRACTICE PAPER 1
25 MARKS

QUESTION 2 · 25 MARKS

A private hospital group with five branches wants a new system built as **microservices**. Key functional requirements: **Patient management** (register patients, medical history, profiles) · **Appointments** (book, reschedule, cancel channelling sessions with doctors) · **Doctors & staff** (schedules, availability, specialities) · **Pharmacy** (prescriptions, stock, dispensing) · **Laboratory** (test orders and results) · **Billing** (invoices, insurance claims, payments) · **Notifications** (SMS reminders, results ready).

2.1 Draw a diagram to depict the microservices scenario. 5 marks

2.2 How can the hospital implement and manage this architecture to achieve scalability, fault isolation and continuous deployment? 5 marks

2.3 Propose a suitable microservices architecture. 5 marks

2.4 What methods can maintain data consistency across the services? 5 marks

2.5 How would you secure patient data and communication between services? 5 marks

3 Paper 1 · Question 3

Smart home controller

PRACTICE PAPER 1
25 MARKS

P1 P2 P3

QUESTION 3 · 25 MARKS

A company builds a **smart home app**. Homes have lights, door locks, air conditioners and cameras from different manufacturers; some speak Zigbee (`zbSwitch(id, on)`), others Bluetooth (`btWrite(mac, cmd)`), and the app can't change the devices' software.

An **"Away mode"** button must turn off all lights, lock every door, set the AC to eco and arm the cameras, all in one tap.

Family **members** can control everything remotely, but **guests** may only switch lights; every remote command must be checked and logged.

Every phone in the house must talk to **one home hub controller**; two controllers would send conflicting commands to the same device.

3.1 Identify the design patterns, with reasons, for: (a) devices with different protocols, (b) the "Away mode" button, (c) member vs guest control, (d) the one home hub controller. — 8 marks

3.2 Draw a class diagram showing how "Away mode" works with the different devices. — 7 marks

3.3 Write Java for the component that stops guests controlling anything except lights. — 5 marks

3.4 Explain which SOLID principles your design follows. — 5 marks

4 Paper 1 · Question 4

Online examination platform

QUESTION 4 · 25 MARKS

A university's **online exam platform** runs every exam through the same process: **authenticate the student → load the paper → conduct the exam → evaluate → publish results**. Evaluation differs by exam type: MCQ exams are auto-marked, essay exams are marked by a lecturer with a rubric, and programming exams run hidden test cases.
Question objects (MCQ, essay, coding) are created depending on what the lecturer adds. When a student submits, the answer script goes through checks in order: **time limit → file format → plagiarism**; more checks will be added later.
Departments may switch on optional proctoring features (webcam monitoring, screen recording, AI flagging) for any exam, in any combination.

4.1 Identify suitable patterns, with reasons, for: (a) the common exam process, (b) creating question objects, (c) submission checks, (d) optional proctoring features. — 8 marks

4.2 Write the abstract class for the exam process using the pattern from (a). — 5 marks

4.3 Write a ProgrammingExam subclass. — 3 marks

4.4 Outline how the submission checks would be implemented so new checks can be added easily. — 5 marks

4.5 Explain how the proctoring features can be added without changing the existing exam classes. — 4 marks

1 Paper 2 · Question 1

Ride-hailing app

PRACTICE PAPER 2
25 MARKS

P1 P2 P3

PRACTICE PAPER 2

INTE 22313 – Software Design Patterns and Frameworks · No. of questions: 4 · Time: 2½ hours
Answer ALL questions. Allocate time by the marks. State any assumptions you make. This is an OPEN BOOK exam.

QUESTION 1 · 25 MARKS

GoLanka is a ride-hailing app. A trip goes from **Requested** → **Driver assigned** → **In progress** → **Completed**. A trip can be **Cancelled** while Requested or Driver assigned, but not once it is in progress. A passenger chooses a vehicle type when booking (**Tuk**, **Car** or **Van**), and each type has its own base fare and capacity.
The final fare is the base fare plus any extras that apply to that trip, in any combination: **surge pricing**, **highway toll**, **night charge**, **extra luggage**. The company does not want a class for every combination.
Surge multipliers and night rates are changed by the operations team during the day, and every part of the system must use the **same current values**.

1.1 Identify suitable design patterns, with reasons, for: (a) trip stages, (b) creating vehicle types, (c) fare extras, (d) the shared current rates. — 8 marks

1.2 Define a Fare interface with getAmount() and getBreakdown(), a BaseFare class, and one extra. — 6 marks

1.3 Draw a state diagram for a trip. — 5 marks

1.4 Explain how the shared rates object is created and used safely when many requests arrive at the same time. — 6 marks

2 Paper 2 · Question 2

Online banking platform

PRACTICE PAPER 2
25 MARKS

P1
P2
P3

QUESTION 2 · 25 MARKS

A bank is replacing its core online banking system with **microservices**. Key functional requirements: **Customer management** (KYC, profiles, login devices) · **Accounts** (savings, current, fixed deposits, balances) · **Transactions** (transfers between accounts and to other banks, bill payments) · **Loans** (applications, approvals, repayment schedules) · **Fraud detection** (real-time checks on transactions) · **Notifications** (SMS/email alerts for every transaction).

2.1 Draw a diagram to depict the scenario. 5 marks

2.2 How can the bank implement and manage microservices to achieve scalability, fault isolation and continuous deployment? 5 marks

2.3 Propose a suitable microservices architecture. 5 marks

2.4 How would you maintain data consistency, especially for money transfers? 5 marks

2.5 How would you secure customer data and service-to-service communication? 5 marks

3 Paper 2 · Question 3

Insurance claim processing

PRACTICE PAPER 2
25 MARKS

P1 P2 P3

QUESTION 3 · 25 MARKS

An insurance company approves claims by amount: an **Agent** can approve up to Rs. 50,000, a **Supervisor** up to Rs. 500,000, a **Manager** up to Rs. 2,000,000, and anything above goes to the **Claims Board**. New approval levels may be added.

Every claim is processed in the same order: **register → assess → approve → pay → notify**. Assessment differs for vehicle, health and property claims (a vehicle claim needs a garage estimate; a health claim needs hospital bills checked; a property claim needs a site visit).

Payments are made through the bank's old mainframe system, which only offers `TRNPAY(String acctNo, long cents, String refCode)`. The claims system expects `pay(Claim c)`.

3.1 Identify the design patterns, with reasons, for: (a) approval by amount, (b) the processing order, (c) the mainframe payment. — 6 marks

3.2 Draw a class diagram for the approval levels. — 6 marks

3.3 Write Java showing how a claim of Rs. 750,000 is processed through the approval levels. — 6 marks

3.4 Every approval level must log the claim, record its decision and log the result. How would you enforce this? — 3 marks

3.5 Show how the claims system can pay through the mainframe without changing either system. — 4 marks

4 Paper 2 · Question 4

Five short cases

PRACTICE PAPER 2
25 MARKS

P1 P2 P3

QUESTION 4 · 25 MARKS

For each case, explain the design pattern you would suggest and why, and give a simple class diagram or code outline.

4.1 A hospital system: patient records must only be opened by doctors and nurses assigned to that patient, and every access must be recorded. — 5 marks

4.2 A reporting module exports the same report as PDF, Excel or CSV, depending on what the user selects; more formats will be added. — 5 marks

4.3 A vending machine behaves differently when it is idle, has a coin inserted, is dispensing, or is out of stock. — 5 marks

4.4 An application sends SMS through an old gateway class with sendText(number, text). A new provider offers dispatch(Message m, int priority). Hundreds of calls to sendText() exist. — 5 marks

4.5 A pizza shop: customers choose a base pizza and add toppings (cheese, chicken, olives, extra sauce) in any combination; the price must be calculated automatically. — 5 marks

1 Paper 3 · Question 1

EV charging station

PRACTICE PAPER 3
25 MARKS

PRACTICE PAPER 3

INTE 22313 – Software Design Patterns and Frameworks · No. of questions: 4 · Time: 2½ hours
Answer ALL questions. Allocate time by the marks. State any assumptions you make. This is an OPEN BOOK exam.

QUESTION 1 · 25 MARKS

A city installs **electric vehicle (EV) charging points**. A charging session can be in these states: **Available**, **Reserved** (a driver booked it in the app), **Charging**, **Payment due** and **Completed**. Any point can also go **Faulty**, and then it accepts nothing until a technician resets it. What the point does when a driver taps "start", "stop" or "pay" depends on its current state.

The charging hardware comes from two vendors that the city cannot modify: Vendor A's charger offers `startCharge(double kW)` / `halt()`; Vendor B's offers `beginSession(int port, String mode)` / `endSession(int port)`. The session software expects `start()` and `stop()`.

1.1 Explain why the State pattern is appropriate for a charging session. — 4 marks

1.2 Draw a state diagram for a charging session. — 5 marks

1.3 Design a UML class diagram for the charging session using the State pattern. — 6 marks

1.4 Write Java for the ChargingSession context class. — 4 marks

1.5 The session software must work with both vendors' chargers. Provide a solution using design pattern knowledge, with code for one vendor. — 6 marks

2 Paper 3 · Question 2

Airline reservation system

PRACTICE PAPER 3
25 MARKS

P1
P2
P3

QUESTION 2 · 25 MARKS

A regional airline is rebuilding its reservation platform as **microservices**. Key functional requirements: **Flight search** (routes, schedules, fares, seat availability) · **Booking** (reserve seats, passenger details, changes and cancellations) · **Payments** (cards, wallets, refunds) · **Check-in** (online check-in, seat selection, boarding passes) · **Loyalty** (miles, tiers, rewards) · **Notifications** (e-tickets, delays, gate changes).

2.1 Draw a diagram to depict the microservices scenario. 5 marks

2.2 How can the airline achieve scalability, fault isolation and continuous deployment? 5 marks

2.3 Propose a suitable microservices architecture. 5 marks

2.4 How would you keep data consistent, for example so that two passengers can't get the same seat? 5 marks

2.5 How would you secure passenger data and communication between services? 5 marks

3 Paper 3 · Question 3

Hotel booking system

PRACTICE PAPER 3
25 MARKS

QUESTION 3 · 25 MARKS

A beach hotel builds a booking system. Rooms are **Standard**, **Deluxe** or **Suite**, and the right room object must be created from the guest's choice.
Guests can add optional extras to a room: **breakfast**, **airport pickup**, **late checkout** and **sea view**, in any combination. The price and description must update automatically, without a class for every combination.
The hotel also sells **holiday packages**. Booking a package must reserve the room, book a spa slot, reserve a restaurant table and schedule the airport transfer. The front-desk screen should do this with one call.

3.1 Identify suitable design patterns, with reasons, for: (a) creating room types, (b) optional extras, (c) booking a package. — 6 marks

3.2 Define a Room interface with getPrice() and getDescription(), and a DeluxeRoom class. — 5 marks

3.3 Draw a class diagram showing how the optional extras are applied. — 6 marks

3.4 Write an outline of the class that books a package with one call. — 4 marks

3.5 Discuss two challenges of the design used for the extras and how you would handle them. — 4 marks

4 Paper 3 · Question 4

Online store returns and refunds

PRACTICE PAPER 3
25 MARKS

QUESTION 4 · 25 MARKS

An online store automates **returns**. A return request is checked in order: **valid receipt → within 14 days → item condition → refund limit**. The checks must be easy to change and extend.
Every refund follows the same steps: **validate the request → calculate the amount → issue the refund → notify the customer**. Issuing the refund differs for **card**, **cash on delivery** (bank transfer) and **store credit**.
For auditing, every refund must be written to **one central audit log** in order, with no duplicate entries. The returns service must keep working even if a server or a data centre fails.

4.1 Identify suitable design patterns, with reasons, for: (a) the return checks, (b) the refund steps, (c) the audit log. — 6 marks

4.2 Write the abstract class for the refund steps, and one subclass. — 5 marks

4.3 Outline how the return checks would be implemented. — 5 marks

4.4 Write the audit log class and explain why it prevents duplicate or out-of-order entries. — 4 marks

4.5 How would you extend the design for High Availability and Disaster Recovery? — 5 marks